

Azure/Entra ID Assessment Full Walkthrough

Commercial-grade offensive assessment methodology

Identity plane | management plane | workloads | M365 | reporting

Masoud Hamada

Offensive Security Analyst

GitHub: [MasoudHamada/notes](#) | YouTube: [@MasoudHamada](#) | Discord: [@MasoudHamada](#)

AZURE

ENTRA ID

GRAPH

RBAC

SENTINEL

DEFENDER

BLOODHOUND





Operational guardrails for authorized Azure / Entra ID assessments



Before You Use This Walkthrough

This document is designed as a professional field guide for authorized Azure, Microsoft Entra ID, Microsoft 365, and cloud identity assessments. It organizes the assessment process into repeatable stages: scoping, unauthenticated discovery, authenticated collection, identity review, management-plane review, workload review, security-operations review, attack-path analysis, and reporting.

Important

All commands and test workflows must be executed only with written authorization, approved scope, rate limits, logging expectations, and agreed evidence-handling rules. The command examples in this document are sanitized and use placeholders for tokens, tenant IDs, usernames, passwords, client secrets, and customer identifiers.

- Use a dedicated assessment workstation and an isolated evidence directory per tenant or subscription.
- Capture evidence as screenshots, command output, exported JSON/CSV, and a finding narrative that maps exposure to business impact.
- Prefer read-only enumeration first. Treat write actions, role assignment changes, password resets, key retrieval, extensions, and exploit validation as explicit-approval activities.
- Do not publish client identifiers, secrets, tenant IDs, tokens, user lists, or screenshots unless they are sanitized.

Table of Contents

Before You Use This Walkthrough

2

Table of Contents

3

1. Assessment Strategy

7

1.1 Engagement Mindset

7

1.2 Evidence Folder Structure

7

1.3 Surface Map

8

2. Tooling and Baseline Collection

8

2.1 Core Tools

8

2.2 Azure CLI and PowerShell Login

9

2.3 Token Collection for Approved Tooling

9

3. BloodHound and AzureHound

9

3.1 What to Collect

9

3.2 Windows AzureHound Workflow

10

3.3 Linux AzureHound Workflow

10

3.4 Useful AzureHound Cypher Queries

10

4. RoadRecon, Dashboards, and Baseline Tools

11

4.1 ROADtools / RoadRecon

11

4.2 ScubaGear

11

4.3 Monkey365

11

4.4 Microsoft Zero Trust Assessment and PowerPipe

11

5. Entra ID Assessment Coverage

12

5.1 Identity and Admin Review

12

5.2 Applications and Enterprise Trust

12

5.3 Azure Management Plane

12

6. Conditional Access, MFA, and Session Controls

13

6.1 Review Objectives	13
6.2 Enumerate Policies	13
6.3 Bypass-Resilience Testing Mindset	13
7. Unauthenticated Enumeration	13
7.1 Tenant and Domain Discovery	13
7.2 Public Asset Discovery	14
7.3 Username Discovery and Validation	14
7.4 Password Spray Validation	14
8. Authentication and Token Handling	14
8.1 Microsoft Graph PowerShell	14
8.2 Azure PowerShell Authentication	15
8.3 Service Principal Authentication	15
8.4 Refresh Tokens and Device Authorization Flow	15
9. General Situational Awareness	16
9.1 Azure CLI Quick Wins	16
9.2 REST / curl with ARM Tokens	16
9.3 Permissions Triage	16
10. Users, Groups, Roles, and Administrative Units	17
10.1 Users	17
10.2 Groups	17
10.3 Directory Roles and PIM	17
10.4 Administrative Units	18
11. App Registrations, Enterprise Applications, and Service Principals	18
11.1 App Registrations	18
11.2 Service Principals	18
11.3 Enterprise Applications	19
12. Devices and Device Trust	19

12.1 Device Review 19

13. Azure Management Plane and RBAC 19

13.1 Hierarchy Review 19

13.2 RBAC Review 19

14. Azure Workload Review 20

14.1 Virtual Machines, Disks, and Snapshots 20

14.2 App Service and Web Apps 20

14.3 Function Apps 21

14.4 Logic Apps and Automation Accounts 21

15. Storage, Key Vault, and Databases 21

15.1 Storage Accounts 21

15.2 Key Vault 22

15.3 SQL, PostgreSQL, and Cosmos DB 22

16. Containers, Registries, and Messaging 22

16.1 Azure Container Registry 22

16.2 Container Instances and Service Bus 23

17. Microsoft 365 and Collaboration Review 23

17.1 Exchange Online 23

17.2 SharePoint, OneDrive, Teams, and Power Platform 23

18. Security Operations Review 24

18.1 Defender for Cloud, Sentinel, and Azure Monitor 24

18.2 KQL Snippets 24

19. Attack-Path Analysis 25

19.1 Build the Chain 25

19.2 Finding Narrative Template 25

20. Reporting and Remediation 26

20.1 Severity Thinking 26

20.2 Executive Summary Pattern 26

20.3 Remediation Roadmap 27

Appendix A - Sanitized Command Index 27

A.1 Data Collection 27

A.2 Identity and App Review 27

A.3 Azure Management and Workloads 27

A.4 KQL 28

Appendix B - Field Checklist 28

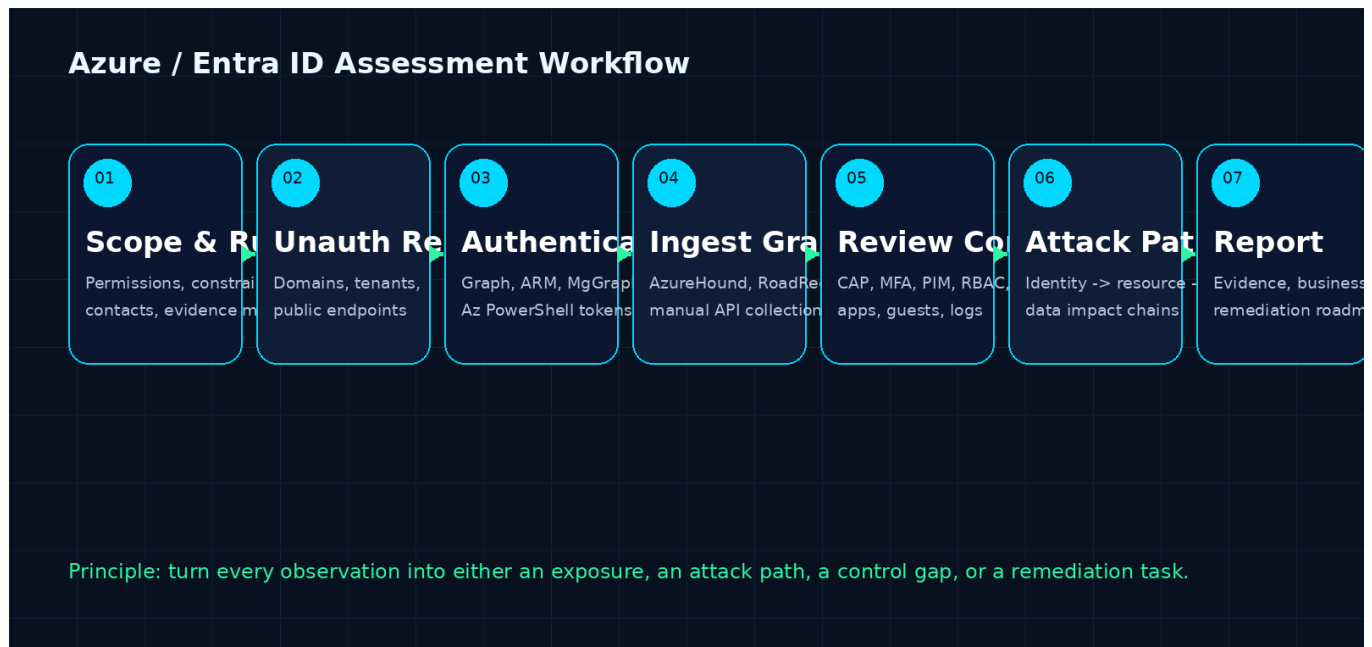
About Masoud Hamada 29

Links and Contact 29

Brand / Platform Logos 29

Public References 30

1. Assessment Strategy



End-to-end assessment flow used throughout this walkthrough.

1.1 Engagement Mindset

The assessment is not just a control checklist. The goal is to understand how identity, permissions, workloads, public exposure, and monitoring combine into realistic attack paths. Every area should be reviewed from two angles: what can be abused, and where the organization can break the chain.

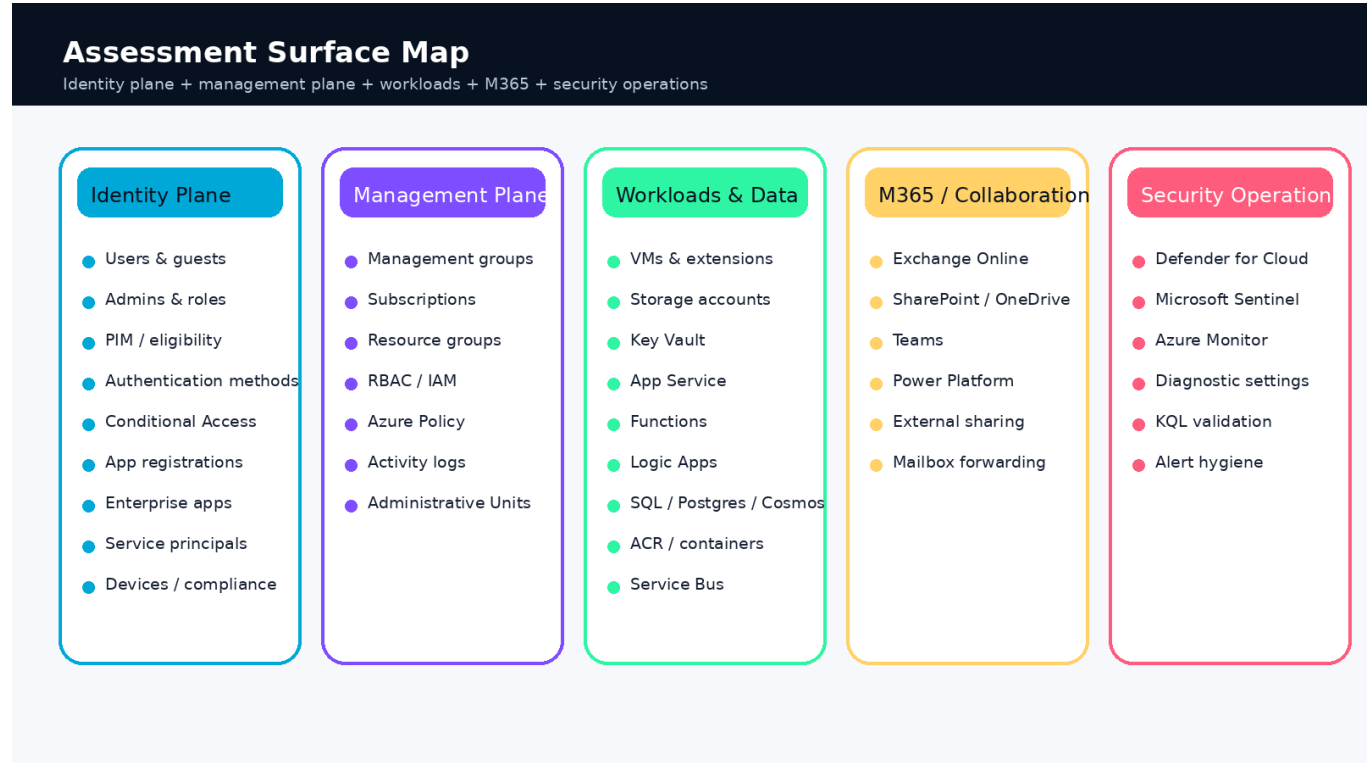
Stage	Goal	Primary Evidence
Scope	Confirm tenant, subscriptions, permission level, IP ranges, no-go actions, and contacts.	Rules of engagement, tenant IDs, approved test users, allowed tooling.
Discovery	Identify tenant names, domains, public endpoints, user formats, and cloud assets.	Unauthenticated recon output, screenshots, DNS/subdomain evidence.
Authenticated collection	Collect Graph, ARM, Entra, subscription, and workload data.	AzureHound/RoadRecon exports, CLI output, PowerShell objects.
Manual validation	Review sensitive defaults, exposures, RBAC, identity controls, applications, and workloads.	Screenshots, JSON/CSV extracts, portal evidence.
Attack-path analysis	Chain weak settings into business impact.	Graph paths, proof-of-concept output, affected identities/resources.
Report	Deliver findings, executive narrative, and prioritized remediation.	Finding writeups, remediation plan, evidence appendix.

1.2 Evidence Folder Structure

Recommended local evidence tree


```
assessment-root/
  00_scope/
  01_unauth_recon/
  02_authentication_tokens/
  03_bloodhound_azurehound/
  04_roadrecon/
  05_entra_identity/
  06_conditional_access_mfa/
  07_rbac_management_plane/
  08_workloads/
  09_m365/
  10_security_ops_logs/
  11_attack_paths/
  12_findings/
  13_sanitized_exports/
```

1.3 Surface Map



Core surfaces to enumerate and correlate during an Azure / Entra assessment.

2. Tooling and Baseline Collection

2.1 Core Tools

Use overlapping tools because each exposes different parts of the tenant. AzureHound and BloodHound are strong for attack-path visualization; ROADtools/RoadRecon is strong for Entra exploration; ScubaGear, Monkey365, Microsoft Zero Trust Assessment, and PowerPipe help baseline control posture.

Tool	Use Case	Output
AzureHound + BloodHound	Graph-based identity and Azure attack path ingestion.	JSON ingestion files and BloodHound attack graphs.
ROADtools / RoadRecon	Entra ID object, app, and policy exploration.	Local database and web GUI.
ScubaGear	Microsoft 365 / tenant security baseline review.	HTML / JSON / CSV reports.

Tool	Use Case	Output
Monkey365	Azure, Entra ID, and M365 posture review.	HTML report and evidence artifacts.
PowerPipe / Steampipe	Dashboards for Azure controls and insights.	Local dashboard server.
Microsoft Graph PowerShell	Manual Graph API review and targeted enumeration.	PowerShell objects and JSON.
Azure PowerShell / Azure CLI	Subscription, RBAC, and resource control-plane review.	CLI/PowerShell output and exports.

2.2 Azure CLI and PowerShell Login

Baseline authentication

```
# Azure CLI interactive login
az login

# Force device-code flow when browser auth is not available
az login --use-device-code

# Azure PowerShell login
Connect-AzAccount

# Microsoft Graph PowerShell login with read scopes
Disconnect-MgGraph
Connect-MgGraph -Scopes
  "User.Read.All","Directory.Read.All","Policy.Read.All","RoleManagement.Read.Directory"
```

2.3 Token Collection for Approved Tooling

Windows-friendly token workflow

```
# Graph token
$GraphToken = az account get-access-token --resource https://graph.microsoft.com --query
  accessToken -o tsv

# ARM token
$ArmToken = az account get-access-token --resource https://management.azure.com/ --query
  accessToken -o tsv

# Tenant ID
$TenantID = az account show --query tenantId -o tsv

# Confirm subscriptions/resources visible to the assessment identity
az account list -o table
az resource list --query "[].{Name:name,Type:type,ResourceGroup:resourceGroup}" -o table
```

Assessment Tip

If Security Reader and Global Reader cannot see subscriptions or resources, the client likely still needs to grant Azure RBAC at the subscription, management group, or resource group scope. Entra roles do not automatically grant Azure resource visibility.

3. BloodHound and AzureHound

3.1 What to Collect

BloodHound should answer: which users, groups, service principals, managed identities, role assignments, subscriptions, resources, and applications create paths to high-value targets? Collect both Graph and ARM perspectives when the scope permits.

- Paths from low-privileged users or guest accounts to privileged roles/resources.
- Paths from service principals of externally facing applications to secrets or management-plane privileges.
- Owners of high-value app registrations, service principals, groups, and subscriptions.
- Overprivileged roles at root, management group, subscription, and resource group scopes.

- High-value targets such as Global Admins, Privileged Role Admins, Owners, Contributor paths, Key Vault access, storage, databases, and automation identities.

3.2 Windows AzureHound Workflow

AzureHound collection on Windows

```
# 1. Login and collect tokens
az login
$TenantID = az account show --query tenantId -o tsv
$GraphToken = az account get-access-token --resource https://graph.microsoft.com --query
  accessToken -o tsv
$ArmToken = az account get-access-token --resource https://management.azure.com/
  --query accessToken -o tsv

# 2. Ingest Graph
cd C:\Tools\Azurehound-v2.8.2\AzureHound_v2.8.2_windows_amd64
.\Azurehound.exe -j $GraphToken --tenant $TenantID list -o Tenant-Graph.json

# 3. Ingest ARM when approved and visible
.\Azurehound.exe -j $ArmToken --tenant $TenantID list -o Tenant-ARM.json

# 4. Open BloodHound CE
.\Bloodhound-cli install
.\Bloodhound-cli config get default_password
# Browse: http://127.0.0.1:8080/ui/login
```

3.3 Linux AzureHound Workflow

AzureHound collection on Linux

```
export TENANT_ID="<>TENANT_ID>"
export GRAPH_TOKEN="$(az account get-access-token --resource https://graph.microsoft.com
  --query accessToken -o tsv)"
export ARM_TOKEN="$(az account get-access-token --resource https://management.azure.com/
  --query accessToken -o tsv)"

/home/pwnedlabs/Tools/BloodHound/AzureHound_v2.7.1_linux_amd64/azurehound -j
  "$GRAPH_TOKEN" --tenant "$TENANT_ID" list -o tenant-GRAPH.json
/home/pwnedlabs/Tools/BloodHound/AzureHound_v2.7.1_linux_amd64/azurehound -j
  "$ARM_TOKEN" --tenant "$TENANT_ID" list -o tenant-ARM.json
```

3.4 Useful AzureHound Cypher Queries

Graph queries for quick path triage

```
// Users and directly connected tenant relationships
MATCH (u:AZUser)-[r]->(t:AZTenant)
RETURN u, t, type(r)
LIMIT 100;

// Users with role relationships
MATCH (u:AZUser)-[r]->(role)
WHERE type(r) CONTAINS "Role"
RETURN u, r, role
LIMIT 100;

// Service principals with role relationships
MATCH (sp:AZServicePrincipal)-[r]->(role)
WHERE type(r) CONTAINS "Role"
RETURN sp, type(r) AS relationship, role
LIMIT 200;

// Groups with no visible members
MATCH (g:AZGroup)
WHERE NOT (g)-[:AZMemberOf]-(:AZUser)
AND NOT (g)-[:AZMemberOf]-(:AZGroup)
RETURN g
LIMIT 100;
```

4. RoadRecon, Dashboards, and Baseline Tools

4.1 ROADtools / RoadRecon

ROADtools install and usage

```
# Install ROADtools
cd ~/Tools
mkdir ROADtools && cd ROADtools
python3 -m venv .venv && source .venv/bin/activate
python -m pip install --upgrade pip setuptools wheel
python -m pip install roadlib roadrecon

# Authenticate and collect
roadrecon auth --device-code
roadrecon gather
roadrecon gui

# Conditional Access policy plugin
roadrecon plugin policies
```

4.2 ScubaGear

ScubaGear baseline

```
# PowerShell
Install-Module -Name ScubaGear -Scope CurrentUser
Import-Module ScubaGear

# Assess all supported products in scope
Invoke-SCuBA -ProductNames *
```

4.3 Monkey365

Monkey365 collection

```
Import-Module C:\AzAD\Tools\monkey365\monkey365.psd1 -Verbose
Invoke-Monkey365 -IncludeEntraID -ExportTo HTML -Instance Azure -ForceAuth -Collect All

# Report location example
C:\AzAD\Tools\monkey365\monkey-reports
```

4.4 Microsoft Zero Trust Assessment and PowerPipe

Dashboard-oriented review

```
# Zero Trust Assessment
pwsh
Import-Module ZeroTrustAssessment
Connect-ZTAssessment
Invoke-ZtAssessment

# PowerPipe / Steampipe dashboards
mkdir dashboards && cd dashboards
powerpipe mod init
powerpipe mod install github.com/turbot/steampipe-mod-azure-compliance
powerpipe mod install github.com/turbot/steampipe-mod-azure-insights
az login
steampipe service start
powerpipe server
```

5. Entra ID Assessment Coverage

5.1 Identity and Admin Review

Area	What to Look For	Why It Matters
Users	Unused users, overprivileged standard users, risky users, sensitive descriptions, stale test accounts, service-style accounts, privileged synced users.	Weak or stale identities create credential attack paths and governance blind spots.
Guests	Guests with broad directory visibility, stale guests, guests in privileged groups, overly permissive B2B trust.	External identities can become internal attack paths if visibility or access is too broad.
Roles and administrators	Too many admins, overuse of Global Administrator, permanent assignments, indirect admin through groups, non-human admins.	Role sprawl increases blast radius and weakens accountability.
PIM	Active instead of eligible roles, no approval, no MFA, long activation duration, no ticketing, no activation alerts.	JIT controls reduce standing privilege and provide better auditability.
Authentication methods	SMS/voice/email OTP allowed, no phishing-resistant MFA for high-value accounts, weak registration controls.	Weak MFA methods create bypass and phishing risk.
Password protection	Audit mode only, weak banned-password posture, sprayable patterns.	Password controls should match tenant risk and identity exposure.

5.2 Applications and Enterprise Trust

Area	Common Finding Patterns
App registrations	Users can register apps unnecessarily; excessive delegated/application permissions; stale/test apps; broad redirect URIs; unclear ownership.
Enterprise applications	Third-party apps with excessive consent; user assignment not required; stale enterprise apps; hidden external integrations.
Service principals	Privileged directory or tenant access; broad resource control; no owner; credentials/certificates without rotation.
Groups	Sensitive groups with unnecessary members; guests in sensitive groups; stale groups; weak owners; broad all-user groups.
Devices	Unmanaged or non-compliant devices accessing sensitive resources; stale devices; privileged users from unmanaged endpoints.
Sign-in and audit logs	Risky sign-ins without response; privileged actions not reviewed; insufficient retention; weak review process.

5.3 Azure Management Plane

Area	Common Finding Patterns
Management groups	High privilege too high in hierarchy; weak inheritance; guardrails not applied at the right level.
Subscriptions	Owner/Contributor sprawl; production and non-production not separated; inconsistent security posture.
Resource groups	Mixed criticality; excessive scope; stale access assignments; weak segregation of duties.
RBAC / IAM	Custom roles too broad; role assignment scope too wide; guests with control-plane access; service principals with broad rights.
Azure Policy	Critical guardrails missing or not enforced; public exposure not prevented; production exemptions.
Activity logs	IAM changes and sensitive resource changes not monitored or alerted.

6. Conditional Access, MFA, and Session Controls

6.1 Review Objectives

Conditional Access should be evaluated by coverage and policy quality. A policy can exist but still leave practical gaps if it excludes admins, guests, workloads, legacy protocols, unmanaged devices, or key SaaS applications.

- No broad MFA policy covering the tenant.
- Coverage gaps for guests, admins, workloads, or important apps.
- Broad exclusions that weaken baseline protection.
- Report-only policies left unimplemented long term.
- Conflicting policy logic that creates inconsistent enforcement.
- Session persistence or sign-in frequency too permissive for sensitive apps.
- Legacy authentication not blocked broadly.

6.2 Enumerate Policies

Conditional Access policy enumeration

```
# Graph beta Conditional Access policy API - requires appropriate permission
az rest --method get --uri
    "https://graph.microsoft.com/beta/identity/conditionalAccess/policies"

# Legacy internal policy endpoint sometimes visible to low-priv users
az rest --method GET --uri
    'https://graph.windows.net/<TENANT_ID>/policies?api-version=1.61-internal' | jq
    '.value[] | select(.policyType == 18) | {displayName, policyDetail: (.policyDetail[]
    | fromjson)}'
```

6.3 Bypass-Resilience Testing Mindset

Authorized Testing Only

The goal is not to evade controls in production. The goal is to prove whether the policy model actually covers approved scenarios and whether the client can detect risky sign-in patterns during a controlled test.

Condition	Review Questions
Network / Named locations	Can sign-in succeed from non-trusted geography, VPN, proxy, or unexpected IP range? Are exclusions too broad?
Device platform	Does the policy rely on user-agent/platform claims that can be varied? Are unmanaged devices blocked where needed?
Client apps	Are browser, mobile/desktop, Exchange ActiveSync, and legacy clients covered appropriately?
Authentication flows	Are device-code and authentication-transfer flows handled according to risk?
Session controls	Are sign-in frequency, persistent browser sessions, and app-enforced restrictions appropriate for sensitive apps?
Risk controls	Do user risk and sign-in risk policies block, require password change, or require strong MFA at the right threshold?

7. Unauthenticated Enumeration

7.1 Tenant and Domain Discovery

Unauthenticated discovery maps the public identity perimeter before any customer credentials are used. Keep this phase low-noise and document exactly which public endpoints were queried.

Tenant and namespace recon

```
# AADInternals outsider recon
Import-Module /home/pwnedlabs/Tools/AADInternals/AADInternals.psd1
Invoke-AADIntReconAsOutsider -DomainName <tenant>.onmicrosoft.com | Format-Table
Get-AADIntTenantID -Domain <domain>
Get-AADIntTenantDomains -Domain <domain>

# User login namespace information
Get-AADIntLoginInformation -UserName <user>@<domain>
```

7.2 Public Asset Discovery

Azure public asset discovery

```
# MicroBurst Azure subdomain enumeration
Import-Module /home/pwnedlabs/Tools/MicroBurst/Misc/Invoke-EnumerateAzureSubDomains.ps1
Invoke-EnumerateAzureSubDomains -Base <company> -Verbose
Get-AzureDomainInfo -folder MicroBurst -Verbose

# cloud_enum example
cloud_enum -k <company> --disable-aws --disable-gcp
```

7.3 Username Discovery and Validation

User discovery and validation

```
# Generate candidate email formats
/opt/gcp_tools/username-anarchy/username-anarchy --suffix @<domain> "First Last" >
  candidate_emails.txt

# Validate usernames - strict authorization and rate limits required
python3 /home/pwnedlabs/Tools/o365creeper-ng/o365creeper-ng.py -f candidate_emails.txt
Get-Content candidate_emails.txt | Invoke-AADIntUserEnumerationAsOutsider -Method Normal

# Teams enumeration using an approved account with Teams access
python3 ./TeamsEnum.py -a password -u <approved_user> -f candidate_emails.txt -o
  teamsenum-output.json
```

7.4 Password Spray Validation

Strict Scope

Password spraying is intrusive. It requires explicit written approval, a lockout-safe plan, client-approved passwords, throttling, logging coordination, and a stop condition.

Authorized spray validation commands

```
# Example company-based password wordlist generation - customize only for approved tests
company="<company>"
for s in 123 1234 2024 2025 Spring Summer Fall Winter Password Password1; do
  for c in "$company" "$(tr '[:lower:]' '[:upper:]' <<< "$company"); do
    echo "${c}${s}"
  done
done > company_passes.txt

# o365spray - approved and rate-limited example
python3 /home/pwnedlabs/Tools/o365spray/o365spray.py --spray -U valid_users.txt -p
  '<PASSWORD>' -d <domain>

# MSOLSpray - approved and rate-limited example
Invoke-MSOLSpray -UserList .\validemails.txt -Password '<PASSWORD>' -Verbose
```

8. Authentication and Token Handling

8.1 Microsoft Graph PowerShell

Microsoft Graph PowerShell authentication

```
# Browser login
Disconnect-MgGraph
Connect-MgGraph

# Device code login with specific scopes
Connect-MgGraph -DeviceCode -Scopes "Directory.Read.All","SecurityAttributes.Read.All"

# Use Azure CLI token to connect to Microsoft Graph PowerShell
$Token = (az account get-access-token --resource https://graph.microsoft.com --query
  accessToken -o tsv)
$SecureToken = ConvertTo-SecureString $Token -AsPlainText -Force
Connect-MgGraph -AccessToken $SecureToken

# Extract current Graph authorization bearer from a MgGraph request
$Parameters = @{ Method = "GET"; Uri = "/v1.0/me"; OutputType = "HttpResponseMessage" }
$Response = Invoke-MgGraphRequest @Parameters
$Headers = $Response.RequestMessage.Headers
$Headers.Authorization.Parameter
```

8.2 Azure PowerShell Authentication

Azure PowerShell login patterns

```
# Interactive browser login
Connect-AzAccount

# Service principal secret login
$appId = "<APP_ID>"
$tenantId = "<TENANT_ID>"
$clientSecret = "<CLIENT_SECRET>"
$secureSecret = ConvertTo-SecureString -String $clientSecret -AsPlainText -Force
$credential = New-Object System.Management.Automation.PSCredential ($appId,
  $secureSecret)
Connect-AzAccount -ServicePrincipal -Credential $credential -Tenant $tenantId

# Connect with ARM and Microsoft Graph access tokens
Connect-AzAccount -AccountId <account_or_object_id> -AccessToken $(cat ArmToken.txt)
  -MicrosoftGraphAccessToken $(cat GraphToken.txt)

# Request access token from current session
(ConvertFrom-SecureString (Get-AzAccessToken -ResourceTypeName Arm
  -AsSecureString).Token -AsPlainText)
(Get-AzAccessToken -Resource "https://graph.microsoft.com").Token
```

8.3 Service Principal Authentication

Service principal auth examples

```
# Azure CLI service principal with secret
az login --service-principal -u <APP_ID> -p '<PASSWORD>' --tenant <TENANT_ID>

# Azure CLI service principal with certificate
az login --service-principal -u <APP_ID> -p '<PASSWORD>' --tenant <TENANT_ID>

# AzureHound with service principal secret
azurehound list -a "<CLIENT_ID>" -s "<CLIENT_SECRET>" -t "<TENANT_ID>" -o ./output.json

# AzureHound with access token / JWT
$JWT=$(az account get-access-token --resource https://graph.microsoft.com -o tsv --query
  accessToken)
azurehound list --jwt "$JWT" -t "<TENANT_ID>" -o ./output.json
```

8.4 Refresh Tokens and Device Authorization Flow

Use Carefully

Device code and refresh token testing should be treated as identity control validation. Coordinate with the client, define the app/client ID under test, and avoid social engineering unless explicitly in scope.

Device authorization flow token collection


```

$ClientID = "<CLIENT_ID>"
$Scope = ".default offline_access"
$body = @{ "client_id" = $ClientID; "scope" = $Scope }
$authResponse = Invoke-RestMethod -UseBasicParsing -Method Post `
    -Uri "https://login.microsoftonline.com/common/oauth2/v2.0/devicecode" -Body $body
$authResponse

# Exchange approved device code for tokens after user completes verification
$GrantType = "urn:ietf:params:oauth:grant-type:device_code"
$body = @{ "client_id" = $ClientID; "grant_type" = $GrantType; "code" =
    $authResponse.device_code }
$Tokens = Invoke-RestMethod -UseBasicParsing -Method Post `
    -Uri "https://login.microsoftonline.com/common/oauth2/v2.0/token" -Body $body
    -ErrorAction SilentlyContinue
$GraphToken = $Tokens.access_token
Write-Output ($Tokens.refresh_token -replace "`r`n", "")

```

9. General Situational Awareness

9.1 Azure CLI Quick Wins

CLI context and resource listing

```

# Account and subscription context
az account show
az account list -o table
az account set --subscription <SUBSCRIPTION_ID>

# List resources
az resource list -o table
az resource list --query "[].{Name:name,Type:type,RG:resourceGroup,Location:location}"
    -o table

# List resource groups
az group list --query "[].{Name:name,Location:location}" -o table

# Current signed-in Entra user
az ad signed-in-user show

```

9.2 REST / curl with ARM Tokens

Direct ARM API examples

```

# List subscriptions with an ARM token
curl -s -H "Authorization: Bearer $(cat ArmToken.txt)"
    "https://management.azure.com/subscriptions?api-version=2020-01-01" | jq

# List resources in a subscription
curl -s -H "Authorization: Bearer $(cat ArmToken.txt)"
    "https://management.azure.com/subscriptions/<SUBSCRIPTION_ID>/resources?api-version=2021-04-01"
    | jq '.value[] | {name,type,resourceGroup,location}'

```

9.3 Permissions Triage

RBAC and role triage

```

# Role assignments visible to current identity
az role assignment list --all -o table

# Role definitions
az role definition list --query
    "[].{Name:roleName,Type:roleType,AssignableScopes:assignableScopes}" -o table

# Current user's object ID
az ad signed-in-user show --query id -o tsv

# Role assignments for a principal
az role assignment list --assignee <OBJECT_ID> --all -o table

```

10. Users, Groups, Roles, and Administrative Units

10.1 Users

User review examples

```
# Graph PowerShell user enumeration
Get-MgUser -All -Property
    Id,DisplayName,UserPrincipalName,UserType,AccountEnabled,CreatedDateTime,OnPremisesSyncEnabled
|
Select-Object
    DisplayName,UserPrincipalName,UserType,AccountEnabled,OnPremisesSyncEnabled,CreatedDateTime

# Risky users - requires appropriate permissions/licensing
Get-MgIdentityProtectionRiskyUser -All |
    Select-Object UserPrincipalName,RiskLevel,RiskState,RiskLastUpdatedDateTime

# Users with descriptions or security attributes should be reviewed for sensitive
information
Get-MgUser -All -Property
    DisplayName,UserPrincipalName>AboutMe,EmployeeId,CreatedDateTime
```

Finding Pattern	Evidence	Remediation Direction
Stale enabled users	Last sign-in / creation date / no owner or business reason.	Disable or delete; enforce lifecycle process.
Overprivileged standard users	Role assignments, group nesting, app ownership.	Move to PIM/JIT and least-privilege groups.
Privileged synced from AD	OnPremisesSyncEnabled = true for high-value admin.	Prefer cloud-only break-glass/admin accounts with strong controls.
Sensitive user attributes	Descriptions/security attributes reveal trust logic or function.	Remove sensitive values and document safe attribute usage.

10.2 Groups

Group review examples

```
# Groups and owners
Get-MgGroup -All -Property
    Id,DisplayName,GroupTypes,SecurityEnabled,MailEnabled,Visibility |
    Select-Object DisplayName,GroupTypes,SecurityEnabled,MailEnabled,Visibility

# Members of a sensitive group
Get-MgGroupMember -GroupId <GROUP_ID> -All

# Owners of a group
Get-MgGroupOwner -GroupId <GROUP_ID> -All

# Groups with no owner are governance risks
Get-MgGroup -All | ForEach-Object {
    $owners = Get-MgGroupOwner -GroupId $_.Id -All
    if (-not $owners) { $_ }
}
```

10.3 Directory Roles and PIM

Role and PIM review

```
# Directory roles currently active
Get-MgDirectoryRole -All | Select-Object Id,DisplayName

# Members of a directory role
Get-MgDirectoryRoleMember -DirectoryRoleId <ROLE_ID> -All

# Role assignment schedules - PIM-aware permission required
Connect-MgGraph -Scopes "RoleAssignmentSchedule.Read.Directory"
Get-MgRoleManagementDirectoryRoleAssignmentSchedule -All

# Role eligibility schedules
Get-MgRoleManagementDirectoryRoleEligibilitySchedule -All
```

10.4 Administrative Units

Administrative Unit review

```
# Enumerate Administrative Units
Connect-MgGraph -Scopes "AdministrativeUnit.Read.All","Directory.Read.All"
Get-MgDirectoryAdministrativeUnit -All | Select-Object Id,DisplayName,Description

# Members of an Administrative Unit
Get-MgDirectoryAdministrativeUnitMember -AdministrativeUnitId <AU_ID> -All

# Scoped role members
Get-MgDirectoryAdministrativeUnitScopedRoleMember -AdministrativeUnitId <AU_ID> -All
```

11. App Registrations, Enterprise Applications, and Service Principals

11.1 App Registrations

App registration review

```
# App registrations with owners and permissions
Get-MgApplication -All -Property
    Id,AppId,DisplayName,SignInAudience,RequiredResourceAccess,Web,PublicClient |
    Select-Object DisplayName,AppId,SignInAudience

# Owners
Get-MgApplicationOwner -ApplicationId <APPLICATION_OBJECT_ID> -All

# Password credentials and certificate credentials metadata
Get-MgApplication -ApplicationId <APPLICATION_OBJECT_ID> -Property
    PasswordCredentials,KeyCredentials |
    Select-Object -ExpandProperty PasswordCredentials
```

11.2 Service Principals

Service principal and consent review

```
# Service principals
Get-MgServicePrincipal -All -Property
    Id,AppId,DisplayName,ServicePrincipalType,AccountEnabled |
    Select-Object DisplayName,AppId,ServicePrincipalType,AccountEnabled

# Owners
Get-MgServicePrincipalOwner -ServicePrincipalId <SP_OBJECT_ID> -All

# App role assignments
Get-MgServicePrincipalAppRoleAssignment -ServicePrincipalId <SP_OBJECT_ID> -All

# OAuth2 permission grants - delegated consent review
Get-MgOAuth2PermissionGrant -All | Select-Object
    ClientId,ConsentType,PrincipalId,ResourceId,Scope
```

11.3 Enterprise Applications

Check	Questions
User assignment required	Can any user access the app without assignment?
Consent model	Who approved permissions and are they still justified?
Owners	Is there a real business owner and technical owner?
High privilege permissions	Does the app hold Directory.ReadWrite.All, AppRoleAssignment.ReadWrite.All, Mail.ReadWrite, Files.ReadWrite.All, or similar?
Stale apps	Has the app been unused but still trusted?
External integrations	Does the third-party vendor trust align with business need?

12. Devices and Device Trust

12.1 Device Review

Device posture examples

```
# Devices
Get-MgDevice -All -Property
    Id, DisplayName, OperatingSystem, TrustType, IsCompliant, IsManaged, ApproximateLastSignInDateTime
|
Select-Object
    DisplayName, OperatingSystem, TrustType, IsCompliant, IsManaged, ApproximateLastSignInDateTime

# Device owners
Get-MgDeviceRegisteredOwner -DeviceId <DEVICE_ID> -All

# Stale device logic example
Get-MgDevice -All -Property
    DisplayName, ApproximateLastSignInDateTime, IsCompliant, IsManaged |
Where-Object { $_.ApproximateLastSignInDateTime -lt (Get-Date).AddDays(-90) }
```

Device review matters because Conditional Access often trusts managed/compliant device claims. Stale or weakly managed devices can become a way for attackers to satisfy access controls that were intended to protect sensitive resources.

13. Azure Management Plane and RBAC

13.1 Hierarchy Review

Management hierarchy commands

```
# Management groups
az account management-group list -o table
az account management-group show --name <MG_NAME> --expand --recurse

# Subscriptions
az account list -o table

# Resource groups
az group list -o table

# Policy assignments
az policy assignment list --scope /subscriptions/<SUBSCRIPTION_ID> -o table
```

13.2 RBAC Review

RBAC review commands

```
# All role assignments in current subscription
az role assignment list --all --query
"[].{Principal:principalName,Type:principalType,Role:roleDefinitionName,Scope:scope}"
-o table

# Role assignments at a specific scope
az role assignment list --scope
/subscriptions/<SUBSCRIPTION_ID>/resourceGroups/<RG_NAME> -o table

# Custom roles
az role definition list --custom-role-only true -o json | jq '.[] | {roleName,
assignableScopes, permissions}'

# Look for high-impact actions in custom roles
az role definition list --custom-role-only true -o json | jq '.[] |
select(.permissions[].actions[]? == "**") | {roleName, assignableScopes}'
```

Risk Pattern	Examples	Break Point
Scope too wide	Owner/Contributor at management group or subscription where resource group would suffice.	Reduce scope; split production/non-production; use PIM.
Service principal overprivileged	Application identity has broad Contributor or Owner.	Create dedicated least-privilege role; rotate credentials.
Guests with control-plane access	External users assigned Owner/Contributor/Reader broadly.	Require business justification, expiration, and conditional access.
Custom role too broad	Actions: * or DataActions: * with broad assignable scope.	Restrict actions and assignable scopes; document purpose.

14. Azure Workload Review

14.1 Virtual Machines, Disks, and Snapshots

VM, extension, disk, and snapshot review

```
# VMs and public IPs
az vm list -d --query
"[].{Name:name,RG:resourceGroup,Power:powerState,PrivateIPs:privateIps,PublicIPs:publicIps,OS:storageProfile.osDisk.
osType}"
-o table

# VM extensions
az vm extension list --vm-name <VM_NAME> --resource-group <RG_NAME> -o table

# Disks
az disk list --query
"[].{Name:name,RG:resourceGroup,DiskState:diskState,OsType:osType,Size:diskSizeGb}"
-o table

# Snapshots
az snapshot list --query
"[].{Name:name,RG:resourceGroup,Time:timeCreated,Size:diskSizeGb}" -o table
```

- Public management ports such as 22 and 3389 exposed to Any.
- Custom script extensions or VM applications that create privilege escalation paths.
- User data/custom data containing secrets.
- Snapshots or disks accessible to identities that do not need them.
- Managed identities attached to VMs with excessive RBAC.

14.2 App Service and Web Apps

App Service review

```
# Web apps
az webapp list --query
"[].{Name:name,RG:resourceGroup,Host:defaultHostName,State:state,HttpsOnly:httpsOnly}"
-o table

# App settings - contains secrets; collect only when approved
az webapp config appsettings list -g <RG_NAME> -n <APP_NAME> -o table

# Auth settings
az webapp auth show -g <RG_NAME> -n <APP_NAME>

# Deployment slots
az webapp deployment slot list -g <RG_NAME> -n <APP_NAME> -o table
```

14.3 Function Apps

Function App review

```
# Function Apps
az functionapp list --query
"[].{Name:name,RG:resourceGroup,Host:defaultHostName,State:state}" -o table

# App settings - secrets likely; explicit approval required
az functionapp config appsettings list -g <RG_NAME> -n <FUNCTION_APP_NAME> -o table

# Function keys - explicit approval required
az functionapp keys list -g <RG_NAME> -n <FUNCTION_APP_NAME>
```

14.4 Logic Apps and Automation Accounts

Logic App and Automation Account review

```
# Logic Apps
az logic workflow list --query
"[].{Name:name,RG:resourceGroup,State:state,Location:location}" -o table
az logic workflow show -g <RG_NAME> -n <WORKFLOW_NAME>

# Automation Accounts
az automation account list --query "[].{Name:name,RG:resourceGroup,Location:location}"
-o table
az automation variable list -g <RG_NAME> --automation-account-name <ACCOUNT_NAME> -o
table
az automation runbook list -g <RG_NAME> --automation-account-name <ACCOUNT_NAME> -o
table
```

15. Storage, Key Vault, and Databases

15.1 Storage Accounts

Storage review

```
# Storage accounts
az storage account list --query
"[].{Name:name,RG:resourceGroup,AllowBlobPublicAccess:allowBlobPublicAccess,PublicNetworkAccess:
publicNetworkAccess}"
-o table

# Containers - requires data-plane permission or account key/SAS
az storage container list --account-name <STORAGE_ACCOUNT> --auth-mode login -o table

# Blob listing - authorized data-plane review only
az storage blob list --account-name <STORAGE_ACCOUNT> --container-name <CONTAINER>
--auth-mode login -o table

# Network rules
az storage account network-rule list -g <RG_NAME> -n <STORAGE_ACCOUNT>
```

Finding Pattern	Impact
Unauthenticated public access enabled	External users may read blobs or static website content without authentication.

Finding Pattern	Impact
SAS tokens broad or long-lived	Token leakage can become direct data exposure.
Storage keys accessible to many identities	Any key reader can bypass fine-grained identity controls.
Metadata contains secrets	Non-obvious secrets in metadata can enable follow-on access.

15.2 Key Vault

Key Vault review

```
# Key Vaults
az keyvault list --query
  "[].{Name:name,RG:resourceGroup,Location:location,PublicNetworkAccess:properties.publicNetworkAccess}"
  -o table

# Access policies
az keyvault show -n <VAULT_NAME> --query properties.accessPolicies

# RBAC assignments at vault scope
VAULT_ID=$(az keyvault show -n <VAULT_NAME> --query id -o tsv)
az role assignment list --scope "$VAULT_ID" -o table

# Secrets - explicit approval required
az keyvault secret list --vault-name <VAULT_NAME> -o table
az keyvault secret show --vault-name <VAULT_NAME> --name <SECRET_NAME>
```

15.3 SQL, PostgreSQL, and Cosmos DB

Database review

```
# Azure SQL servers and databases
az sql server list --query
  "[].{Name:name,RG:resourceGroup,FQDN:fullyQualifiedDomainName,PublicNetworkAccess:publicNetworkAccess}"
  -o table
az sql db list -g <RG_NAME> -s <SQL_SERVER> -o table
az sql server firewall-rule list -g <RG_NAME> -s <SQL_SERVER> -o table

# PostgreSQL flexible servers
az postgres flexible-server list --query
  "[].{Name:name,RG:resourceGroup,State:state,PublicNetworkAccess:network.publicNetworkAccess}"
  -o table

# Cosmos DB accounts
az cosmosdb list --query
  "[].{Name:name,RG:resourceGroup,PublicNetworkAccess:publicNetworkAccess,Kind:kind}"
  -o table
```

16. Containers, Registries, and Messaging

16.1 Azure Container Registry

ACR review

```
# Registries
az acr list --query
  "[].{Name:name,RG:resourceGroup,LoginServer:loginServer,AdminUser:adminUserEnabled,PublicNetworkAccess:
publicNetworkAccess}"
  -o table

# Repositories - requires permission
az acr repository list -n <ACR_NAME> -o table

# Role assignments at registry scope
ACR_ID=$(az acr show -n <ACR_NAME> --query id -o tsv)
az role assignment list --scope "$ACR_ID" -o table
```

16.2 Container Instances and Service Bus

Container instance and Service Bus review

```
# Container instances
az container list --query
"[].{Name:name,RG:resourceGroup,IP:ipAddress.ip,State:instanceView.state}" -o table

# Service Bus namespaces
az servicebus namespace list --query
"[].{Name:name,RG:resourceGroup,Location:location,PublicNetworkAccess:publicNetworkAccess}"
-o table
az servicebus queue list -g <RG_NAME> --namespace-name <NAMESPACE> -o table
az servicebus topic list -g <RG_NAME> --namespace-name <NAMESPACE> -o table
```

17. Microsoft 365 and Collaboration Review

17.1 Exchange Online

Exchange findings often map directly to data exfiltration, persistence, and business email compromise risk. External forwarding and mailbox rules should be treated as high-signal review areas.

Exchange forwarding review

```
# Connect to Exchange Online
Connect-ExchangeOnline

# Mailbox forwarding
Get-Mailbox -ResultSize Unlimited | Select-Object
  DisplayName,UserPrincipalName,ForwardingAddress,ForwardingSmtpAddress,DeliverToMailboxAndForward

# Inbox rules with forwarding
Get-Mailbox -ResultSize Unlimited | ForEach-Object {
  Get-InboxRule -Mailbox $_.UserPrincipalName | Where-Object { $_.ForwardTo -or
    $_.RedirectTo -or $_.ForwardAsAttachmentTo }
}
```

17.2 SharePoint, OneDrive, Teams, and Power Platform

Area	Review Focus
SharePoint / OneDrive	Public or anonymous sharing, stale links, link expiration, external sharing defaults, sensitive sites.
Teams	Guest participation, external collaboration, app/plugin permissions, sensitive data shared in channels.
Power Platform	Unrestricted trials, broad self-service app creation, connectors with sensitive data, environment sprawl.
GraphRunner-assisted search	Authorized search for terms such as password, key, secret, token, bonus, finance, payroll, or client names.

GraphRunner search examples

```
# GraphRunner examples - authorized data review only
Import-Module /home/pwnedlabs/Tools/GraphRunner/GraphRunner.ps1
Get-GraphTokens
$tokens.access_token

Invoke-SearchSharePointAndOneDrive -Tokens $tokens -SearchTerm 'password'
Invoke-SearchTeams -Tokens $tokens -SearchTerm 'password'
Invoke-SearchMailbox -Tokens $tokens -SearchTerm "password" -MessageCount 40
```


18. Security Operations Review

18.1 Defender for Cloud, Sentinel, and Azure Monitor

Control Area	What to Validate
Defender for Cloud	Alerts remain unevaluated; unhealthy resources; Defender plans missing for important workloads; recommendations ignored.
Microsoft Sentinel	Critical detections not enabled; logs not onboarded; no detection path for privileged or exposure-related events.
Azure Monitor / logging	Diagnostic settings missing; log retention too short; high-impact changes not alerting; inconsistent resource visibility.
Activity logs	IAM changes, role assignments, policy changes, and sensitive resource changes should be traceable and reviewed.

18.2 KQL Snippets

KQL review snippets

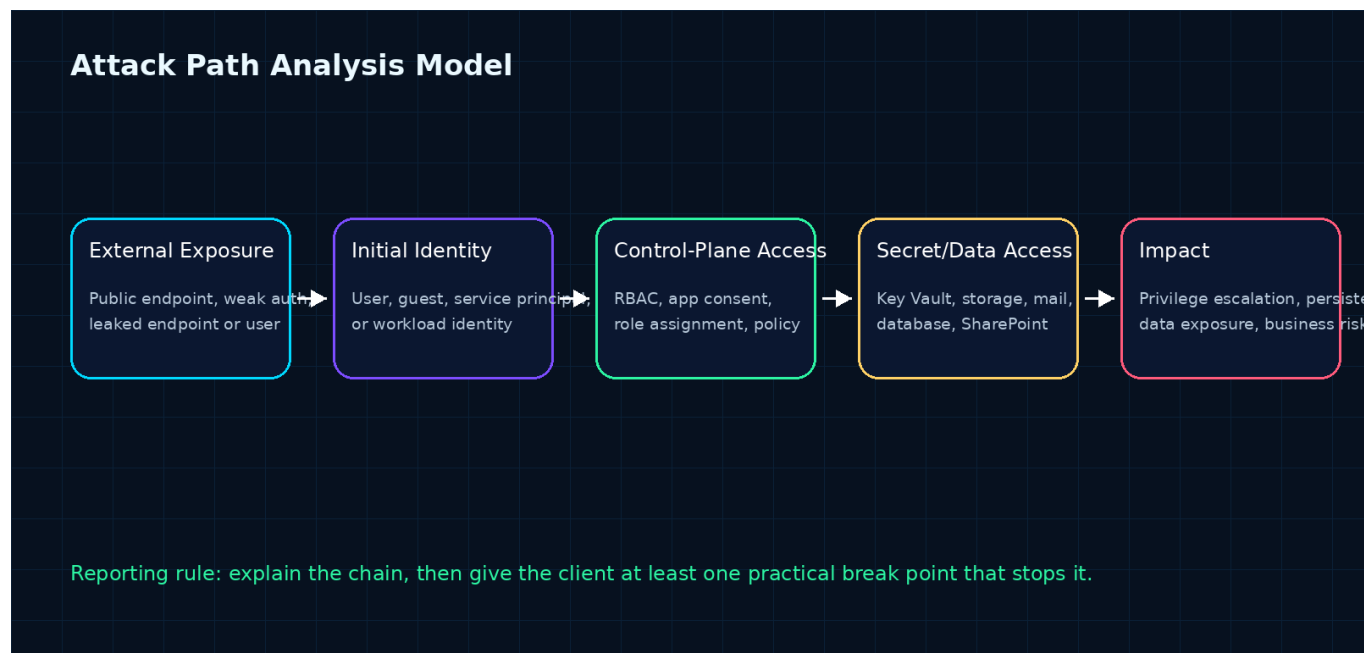
```
// Successful logins in the last hour
SigninLogs
| where TimeGenerated > ago(1h)
| where ResultType == 0
| summarize Count=count() by UserPrincipalName, IPAddress, AppDisplayName
| sort by Count desc

// Unsuccessful logins
SigninLogs
| where TimeGenerated > ago(24h)
| where ResultType != 0
| summarize Count=count() by UserPrincipalName, IPAddress, ResultDescription
| sort by Count desc

// Privileged role changes
AuditLogs
| where TimeGenerated > ago(30d)
| where Category =~ "RoleManagement"
| project TimeGenerated, ActivityDisplayName, InitiatedBy, TargetResources, Result

// Storage account changes
AzureActivity
| where TimeGenerated > ago(30d)
| where ResourceProviderValue =~ "MICROSOFT.STORAGE"
| project TimeGenerated, Caller, OperationNameValue, ResourceGroup, Resource
```

19. Attack-Path Analysis



Generic model for translating cloud observations into business-impact attack paths.

19.1 Build the Chain

A strong cloud finding is not just a misconfiguration. It is a chain: who can reach the weakness, what identity or data it exposes, which permissions become available, what the business impact is, and which control would break the chain.

Question	Example Evidence
Who can access the starting point?	Public endpoint, guest, low-privileged user, service principal, workload identity, leaked SAS.
What does the starting point expose?	Token, managed identity, app setting, secret, storage blob, mail data, database, code, deployment endpoint.
What privilege can be gained?	RBAC assignment, Graph permission, app owner, Key Vault secret reader, storage key reader, subscription contributor.
What is the impact?	Data access, persistence, privilege escalation, tenant compromise, operational disruption.
Where can the chain be broken?	Private endpoint, RBAC least privilege, PIM, CAP, secret rotation, disable public access, logging/alerting.

19.2 Finding Narrative Template

Reusable cloud finding template

```

Finding Title:
<Identity / Azure / M365 control gap expressed as a clear risk>

Affected Assets:
<Users, groups, app IDs, resource IDs, subscriptions, storage accounts, vaults, apps>

Attack Path:
1. <Starting condition>
2. <Action an attacker can take>
3. <Privilege, secret, or data obtained>
4. <Business impact>

Evidence:
- Screenshot: <file>
- Command output: <file>
- Export: <JSON/CSV>

Impact:
<Explain business risk in language leadership understands.>

Remediation:
<Prioritized technical steps and governance/process fixes.>

Validation:
<How the client can confirm remediation worked.>

```

20. Reporting and Remediation

20.1 Severity Thinking

Severity	Cloud / Identity Interpretation
Critical	Direct path to tenant-wide admin, subscription Owner, sensitive data at scale, or high-impact persistence with low complexity.
High	Compromise of important business data, privileged app/service principal, production workload, or realistic path to escalation.
Medium	Meaningful exposure or control gap requiring additional conditions, limited data, or constrained privilege.
Low	Hardening issue, incomplete governance, low-impact exposure, or weak configuration with limited immediate abuse.
Informational	Context, positive observations, hygiene notes, or future improvement opportunities.

20.2 Executive Summary Pattern

Executive summary starter

During the Azure and Microsoft Entra ID assessment, we identified multiple identity and cloud configuration weaknesses that could allow an attacker to move from low-trust access into sensitive cloud resources. The highest-risk issues involved <top issue 1>, <top issue 2>, and <top issue 3>. These weaknesses increased the blast radius of compromised identities and reduced the organization's ability to contain identity-based attacks.

The recommended remediation priority is to reduce standing privilege, enforce stronger Conditional Access and MFA controls for privileged workflows, restrict public exposure, tighten RBAC scopes, rotate exposed secrets, and improve logging coverage for high-impact changes.

20.3 Remediation Roadmap

Priority	Remediation Theme	Examples
0-7 days	Contain high-risk paths	Disable public access, remove exposed secrets, revoke broad tokens/SAS, rotate credentials, remove unnecessary Owners.
7-30 days	Reduce identity blast radius	Enable PIM, convert active roles to eligible, enforce phishing-resistant MFA for admins, restrict guests.
30-60 days	Harden management plane	Tighten RBAC scopes, enforce Azure Policy, private endpoints for sensitive services, Key Vault network controls.
60-90 days	Operationalize detection	Diagnostic settings, Sentinel analytics, Defender recommendations, alerting for IAM and high-impact resource changes.
Ongoing	Governance	Access reviews, app consent reviews, guest lifecycle, stale resource cleanup, quarterly attack-path reviews.

Appendix A - Sanitized Command Index

This appendix consolidates high-value command patterns from the walkthrough. Replace placeholders with approved scope values only. The original notes contained lab-specific values and credentials; those were intentionally sanitized.

A.1 Data Collection

```
# AzureHound quick collection
$TenantID = az account show --query tenantId -o tsv
$GraphToken = az account get-access-token --resource https://graph.microsoft.com --query accessToken -o tsv
$ArmToken = az account get-access-token --resource https://management.azure.com/ --query accessToken -o tsv
. AzureHound.exe -j $GraphToken --tenant $TenantID list -o Tenant-Graph.json
. AzureHound.exe -j $ArmToken --tenant $TenantID list -o Tenant-ARM.json

# RoadRecon
roadrecon auth --device-code
roadrecon gather
roadrecon gui

# Monkey365
Invoke-Monkey365 -IncludeEntraID -ExportTo HTML -Instance Azure -ForceAuth -Collect All
```

A.2 Identity and App Review

```
Get-MgUser -All -Property Id,DisplayName,UserPrincipalName,UserType,AccountEnabled,CreatedDateTime,OnPremisesSyncEnabled
Get-MgGroup -All -Property Id,DisplayName,GroupTypes,SecurityEnabled,MailEnabled,Visibility
Get-MgApplication -All -Property Id,AppId,DisplayName,SignInAudience,RequiredResourceAccess
Get-MgServicePrincipal -All -Property Id,AppId,DisplayName,ServicePrincipalType,AccountEnabled
Get-MgOAuth2PermissionGrant -All
Get-MgDevice -All -Property Id,DisplayName,OperatingSystem,TrustType,IsCompliant,IsManaged,ApproximateLastSignInDateTime
```

A.3 Azure Management and Workloads

```
az account list -o table
az resource list -o table
az role assignment list --all -o table
az role definition list --custom-role-only true -o json
az vm list -d -o table
az webapp list -o table
az functionapp list -o table
az storage account list -o table
az keyvault list -o table
az sql server list -o table
az acr list -o table
az servicebus namespace list -o table
```

A.4 KQL

```

SigninLogs
| where TimeGenerated > ago(24h)
| summarize Count=count() by UserPrincipalName, IPAddress, ResultType, AppDisplayName
| sort by Count desc

AuditLogs
| where TimeGenerated > ago(30d)
| where Category =~ "RoleManagement"
| project TimeGenerated, ActivityDisplayName, InitiatedBy, TargetResources, Result

AzureActivity
| where TimeGenerated > ago(30d)
| where OperationNameValue has_any ("roleAssignments/write", "policyAssignments/write", "vaults/write")
| project TimeGenerated, Caller, OperationNameValue, ResourceGroup, Resource

```

Appendix B - Field Checklist

Domain	Checklist
Entra users	Admins count; stale accounts; privileged synced users; risky users; weak descriptions/attributes; guest default.
Conditional Access	Coverage for admins/users/guests/workloads; report-only policies; legacy auth; device controls; session controls.
PIM	Eligible vs active; approval; MFA; justification; ticketing; activation duration; notifications; access reviews.
Apps and SPs	Owners; credentials; high-risk Graph permissions; app role assignments; stale apps; user assignment required.
RBAC	Owner/Contributor sprawl; scope too high; custom roles; guest access; service principal role assignments.
Storage and Key Vault	Public access; SAS; keys; secrets; public network; private endpoints; rotation.
Workloads	Public endpoints; managed identities; extensions; app settings; deployment slots; workflows; automation variables.
Security ops	Defender plans; Sentinel onboarded logs; diagnostic settings; alerting for IAM/resource changes; retention.
Reporting	Attack path; affected assets; business impact; evidence; remediation; validation plan.

About Masoud Hamada



Masoud Hamada
Offensive Security Analyst

Offensive Security Analyst focused on offensive security operations, cloud and identity assessments, adversary simulation, and practical security reporting. Experience includes Azure, AWS, GCP, Active Directory, Entra ID, IAM abuse paths, cloud exploitation, detection evasion, and attack-path-driven reporting.

Current / Recent Roles: Secured Approach - Offensive Security Analyst; ILLUMANT - Offensive Security Analyst; Pwned Labs - Offensive Security Analyst.

Education: MacEwan University - Bachelor of Commerce, Business Administration and Management.

Selected Certifications / Training: CRT0, CRTP, M-CRTP3, MCRTTP, GCRTTP, OSCP in progress, plus additional cloud/red-team training and public lab notes.

Community: Cybersecurity content creator, mentor, lab walkthrough author, and public note publisher.

Links and Contact

	GitHub https://github.com/MasoudHamada/notes		LinkedIn https://www.linkedin.com/in/masoud-hamada/
	YouTube https://www.youtube.com/@MasoudHamada		Discord @MasoudHamada

Brand / Platform Logos

The visual branding below is represented as clean typographic logo badges for the platforms, companies, and certification bodies referenced in the profile and walkthrough.

Microsoft Azure	Microsoft Entra ID	Microsoft Graph
BloodHound	GitHub	LinkedIn
YouTube	Discord	Secured Approach
ILLUMANT	Pwned Labs	MacEwan University
Zero-Point Security	OffSec	Hack The Box

Public References

Reference	URL
GitHub Notes Repository	https://github.com/MasoudHamada/notes
LinkedIn Profile	https://www.linkedin.com/in/masoud-hamada/
YouTube Channel	https://www.youtube.com/@MasoudHamada
Microsoft Entra ID On-Demand Assessment documentation	https://learn.microsoft.com/en-us/services-hub/unified/health/getting-started-entraid
AzureADAssessment repository	https://github.com/AzureAD/AzureADAssessment